

Problem 1 — Decoy-state parameter estimation for Twin-Field QKD (simulation + finite-size)

Scope. Analyze and implement decoy-state methods for Twin-Field (TF) QKD under realistic imperfections and finite-size effects. Compare how different decoy-state choices affect secure key rate and practical parameter estimation.

Objectives

- * Derive/implement finite-size decoy-state parameter-estimation formulas tailored to a TF-QKD protocol.
- * Simulate performance (secure key rate vs distance and block size) for common decoy choices (e.g., 2-decoy, 3-decoy, optimized intensities).
- * Quantify sensitivity to intensity fluctuation, detector inefficiency, and phase-noise.

Suggested approach

- * Start from a baseline TF rate equation (as in original TF proposals) and add decoy estimation steps.
- * Implement Monte-Carlo finite-size simulations in Python/Matlab (simulate photon number statistics, losses, detection).
- * Compare analytical bounds vs numerical bootstrapping for confidence intervals.
- * Produce plots: key rate vs distance for various decoy strategies and finite block sizes.

Prerequisites

- * Basic QKD theory, probability/statistics, Python/Matlab/Mathematica coding.

Deliverables

- * Well-commented code + simulation notebook.
- * Short report (6–8 pages) with derivations, plots, and recommendations for practical decoy settings.
- * 12–15 min presentation + poster.

8-week milestone

- * Weeks 1–2: literature & derivations; choose decoy strategies.
- * Weeks 3–5: implement simulator and parameter estimation.
- * Week 6: sensitivity analyses.
- * Weeks 7–8: write report, prepare presentation.

Problem 2 — Comparative analysis of Twin-Field QKD variants: operational and security subtleties

Scope. Systematically study and explain subtle differences between multiple TF-QKD variants (original TF, phase-matched/PM-QKD, sending-or-not-sending (SNS) variants, and practical implementations). Provide a unified comparison targeting experimental feasibility and security assumptions.

Objectives

- * Produce a clear taxonomy of TF-type protocols, listing differing assumptions (phase-reference requirements, postselection, encoding/decoding methods, device trust model).
- * Analyze how the differences map to experimental constraints (phase stabilization, source requirements) and to security proofs (what's assumed/relaxed).
- * Provide a short guide recommending which variant suits which experimental scenario.

Suggested approach

- * Conduct a focused literature review and extract a checklist of assumptions for each variant.
- * Build a small simulation or tabulated comparison (metrics: key rate vs loss, required phase stability, resilience to intensity mismatch).
- * Highlight cases where one variant's security proof does not trivially imply security for another (point out hidden assumptions).

Prerequisites

* Familiarity with QKD security proofs and basic TF protocol structure; comfortable reading research papers.

Deliverables

- * Comparative technical report (8–10 pages) with conceptual diagrams and a decision flowchart for experimentalists.
- * A short “cheat-sheet” (1–2 pages) that experimental students can use to pick a TF variant.
- * Presentation (10–12 min) and Q&A notes.

8-week milestone

- * Weeks 1–2: literature review & taxonomy.
- * Weeks 3–5: formalize differences and run basic simulations / examples.
- * Weeks 6–8: finalize report, cheat-sheet, presentation.

Problem 3 — AI/ML for adaptive stabilization and anomaly detection in QKD sessions

Scope. Design and evaluate ML tools that assist QKD sessions: (a) predict and correct phase/drift for better interference visibility, and/or (b) detect anomalies (hardware faults or attack patterns) from side-channel.

Objectives

- * Build a dataset (simulated) containing phase drift, counts, timing jitter, intensities, environmental variables.
- * Train ML models (time-series: LSTM/GRU; or classical regressors) to predict near-term phase drift and propose adaptive corrections.
- * Alternatively/additionally develop a classifier to detect anomalous sessions or signatures of tampering/attacks.
- * Measure improvement: interference visibility, QBER reduction, or reduction in false alarms.

Suggested approach

- * Generate synthetic data from simulations with realistic noise and drift models; augment with randomized attack scenarios.
- * Baseline models: compare performance and computational cost.
- * Integrate model outputs into a simulated QKD control loop to observe impact on secure key rate or QBER.

Prerequisites

* Basic ML (supervised learning, time-series), Python (scikit-learn, PyTorch/TensorFlow), QKD operational knowledge.

Deliverables

- * Simulated dataset (or data generation scripts), trained models, and inference code.
- * Evaluation report comparing ML vs classical control; practical recommendations for deployment.
- * Demo notebook showing closed-loop simulation + presentation.

8–10 week milestone

- * Weeks 1–2: dataset collection/generation & baseline models.
- * Weeks 3–6: train/validate models and run closed-loop simulations.
- * Weeks 7–8(10): robustness tests, write-up, demo & presentation.
